Week 2 Level 3: Advanced

Objects: Blueprints and Things

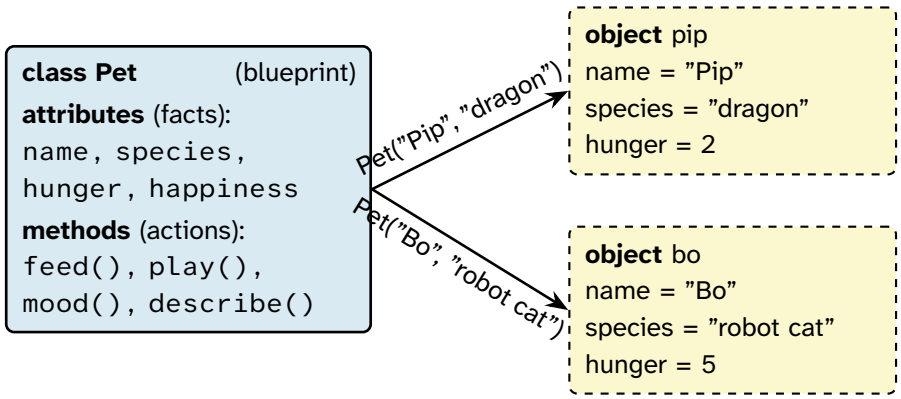
A class is a blueprint; objects are the things you build from it, and each one keeps its own data.

What you will learn

- ☐ Write a class with `__init__`, attributes and methods.
- ☐ Make many objects from one class and change one without changing the others.
- ☐ Use `self` correctly (and fix code that forgets it).
- ☐ Paint turtle art with a list of objects.

The big idea

A **class** is a blueprint, like a cookie cutter. An **object** is a thing made from the blueprint, like one cookie. Every object made from the same class has the same *kinds* of facts, but its own *values*.



Anatomy of a class

Every part has a job

```
class Pet:                          # 1. the blueprint (capital P)
    def __init__(self, name):       # 2. setup: runs when a pet is made
        self.name = name            # 3. attributes: facts it remembers
        self.hunger = 5
```

```
    def feed(self):                    # 4. a method: an action
        self.hunger = self.hunger - 3   # MY hunger goes down

pip = Pet("Pip")                       # 5. make an object
pip.feed()                             # 6. ask it to do something
print(pip.hunger)                      #    prints 2
```

Tip: What is self?

`self` means "this object" or "my". When you write `pip.feed()`, Python runs `feed` with `self` set to `pip`. So `self.hunger` is Pip's hunger. Every method needs `self` as its first parameter, and every attribute is written `self.something`.

From dictionary to class

In Level 2 you might have written `pip = {"name": "Pip", "hunger": 5}`. A dictionary holds the facts. A class holds the facts *and* the actions, all in one place, so you can make 100 pets that all know how to `feed()`.

Exercise 2.1: Pet Class

Folder: `week02/ex-pet-class`. `__init__` and `feed` are done. Finish:

- `play()`: happiness $+2$ (max 10) and hunger $+1$ (max 10).
- `mood()`: `"hungry"` if hunger ≥ 7, else `"happy"` if happiness ≥ 7, else `"bored"` if happiness ≤ 3, else `"okay"`.
- `describe()`: returns `"Pip the dragon feels okay."`

Predict first: after `pip.play()` twice, what are Pip's hunger and happiness? _____

What is Bo's hunger? _____

Exercise 2.2: Bubble Art

Folder: `week02/ex-bubble-art`. Each `Bubble` object remembers `x`, `y`, `size` and `color`. Finish `__init__`, `grow`, `move`, the big dot in `draw`, and `make_bubbles`, which returns a **list of objects**:

```
bubbles = make_bubbles(40)      # a list of 40 Bubble objects
for bubble in bubbles:
    bubble.draw(pen)            # each one draws itself
```

Done looks like: a black window full of bubbles, each with a white shine. Check with `check_my_`

`work.py`, then commit your picture's code in `git-quest`.

Tip: Kind design

Our pets never get sick, run away or say "you forgot me!". Good games are fun because you want to play, not because they make you feel guilty. When you design a game, ask: "Is this fair and kind to the player?"

Vocabulary box

class A blueprint that says what facts and actions a kind of thing has.

object One thing made from a class, with its own values. Also called an instance.

attribute A fact an object remembers, like `self.hunger`.

method A function that lives inside a class, like `feed(self)`.

__init__ The setup method ("dunder init"). It runs when a new object is made.

self Inside a method: "the object that is running this code", or "my".

Reflection

Pick something from a game you like. Write it as a class: its name, 3 attributes and 2 methods.

What does `self` mean, in your own words?

Week 2 Level 3: Advanced

Objects: Blueprints and Things

15–20 minutes. Do Core first; Challenge and Creative are optional.

Name (nickname is fine): _____ Date: _____

★ Core (everyone): Coin Purse (about 10 minutes)

1. Open `week02/hw-coin-purse/starter.py` in Thonny. A `CoinPurse` object keeps a player's coins.
2. Finish `add` (ignore `0` or negative amounts), `can_afford` and `spend` (only pay if you can afford it; return `True` or `False`).
3. Run it. The last line should be `Found 7 coins! Now: 9 coins.`
4. Open `check_my_work.py` and press Run until all checks pass.
5. Copy your file into `Documents/git-quest` and commit it with a helpful message.

Predict: `purse = CoinPurse(10)`, then `purse.spend(4)`, then `purse.spend(7)`. What does the second `spend` return, and how many coins are left? _____

★★ Challenge (want more?): Design a class on a card

Choose a thing from a game: a spaceship, a plant, a door, a monster. Fill in its class card. Then, if you have time, write it in Python with `__init__` and two methods, and make two objects.

Class name	
Attributes (facts)	
Methods (actions)	
Two objects	

Creative (make it yours): Make the bubbles yours

Add a new method to your `Bubble` class from class, for example `pop(self, t)` that draws a ring of 8 small dots around the bubble, or `rainbow(self, t)` that draws 3 smaller dots inside it in different palette colors. Use it on some of the bubbles. Sketch your picture here:

Self-check: tick each box when it is true

- ☐ All checks pass in `hw-coin-purse`.
- ☐ Every method I wrote has `self` as its first parameter.
- ☐ I can explain the difference between a class and an object.
- ☐ I committed my work in `git-quest` with a helpful message.